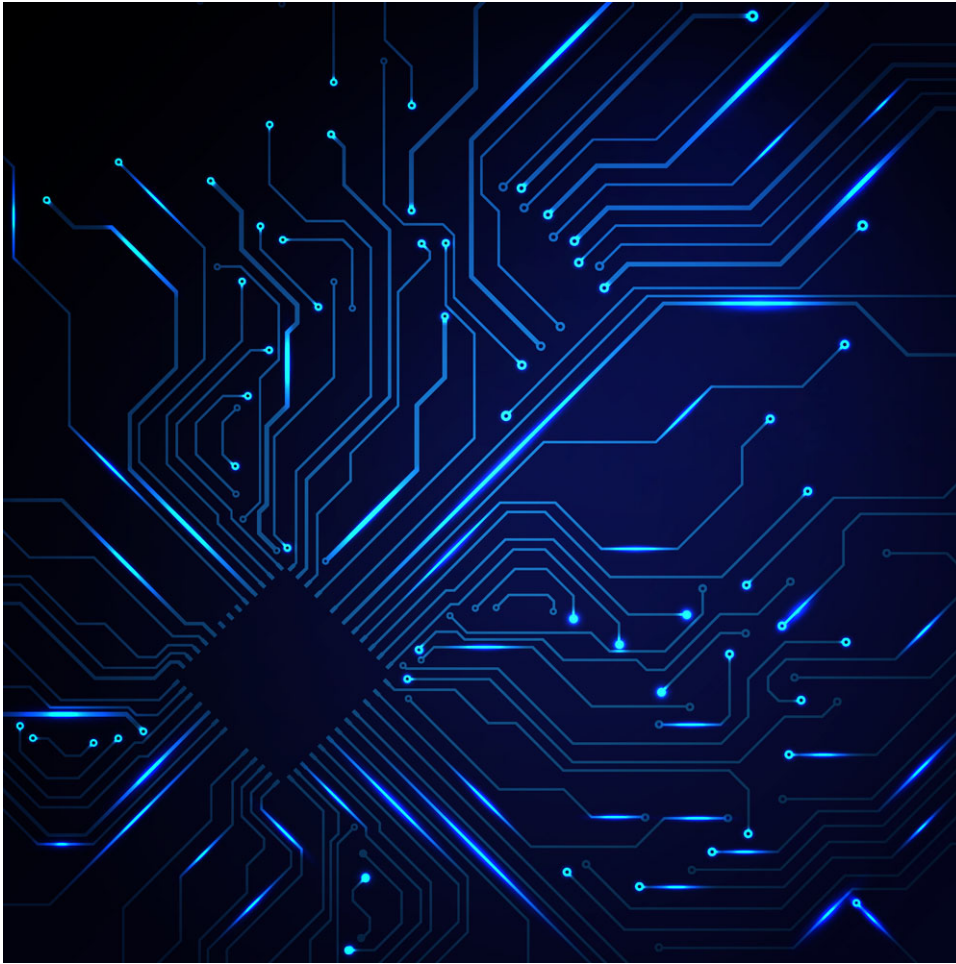




LAB 9 SEQUENTIAL LOGIC PART 2

Name : 01357101

Student ID : 黃翊宏

Note

1. All files must be converted to PDF format before submission.
2. **Submission deadline:**
Next Tuesday at 1:00 PM
3. Each person submits one assignment.



Lab 9-1 :

Main Code Screenshot

updown.v

```
V updown.v X
de0_empty > design > V updown.v
1 module updown(
2     input logic clk,
3     input logic reset,
4     input logic x,
5     output logic [3:0]Q
6 );
7     logic a, b, c, d, Da, Db, Dc, Dd;
8
9     dfflipflop diff1 ( .d(Da), .clk(clk), .reset(reset), .q(a));
10    dfflipflop diff2 ( .d(Db), .clk(clk), .reset(reset), .q(b));
11    dfflipflop diff3 ( .d(Dc), .clk(clk), .reset(reset), .q(c));
12    dfflipflop diff4 ( .d(Dd), .clk(clk), .reset(reset), .q(d));
13
14    assign Dd = ~d;
15    assign Dc = (~x & (c ^ d)) | (x & (~c & ~d | c & d));
16    assign Db = (~x & (b & (~c | ~d) | ~b & c & d)) | (x & (b & (c | d) | ~b & ~c & ~d));
17    assign Da = (~x & (a & ~b | a & ~c | a & ~d | ~a & b & c & d)) | (x & (a & b | a & c | a & d | ~a & ~b & ~c & ~d));
18
19    assign Q = {a, b, c, d};
20 endmodule
```

dfflipflop.v

```
V updown.v V dfflipflop.v X
de0_empty > design > V dfflipflop.v
1 module dfflipflop(
2     input d,
3     input clk,
4     input reset,
5     output logic q,
6     output logic q_n
7 );
8     //正邊緣觸發 D型正反器
9     always_ff @(posedge clk)
10    begin
11        if (reset) begin
12            q <= 1'b0;
13            q_n <= 1'b1;
14        end
15        else begin
16            q <= d;
17            q_n <= ~d;
18        end
19    end
20
21 endmodule
```

freg_div_23.v

```
V updown.v V dfflipflop.v V freg_div_23.v X
de0_empty > design > V freg_div_23.v
1 module freg_div_23 (
2     input logic clk,
3     input logic reset,
4     output logic clk_out
5 );
6
7     logic [31:0] q;
8
9     always_ff @(posedge clk or posedge reset)
10    begin
11        if (reset)
12            q <= 0;
13        else
14            q <= q+1;
15    end
16
17    assign clk_out = q[23];
18 endmodule
```

Karnaugh Map and Derivation Results (D_A D_B D_C D_D)

$X=0$

A:

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	1	1	0	1
10	1	1	1	1

$$A(t+1)_1 = A'BCD + AD' + AC' + AB'$$

B:

AB \ CD	00	01	11	10
00	0	0	1	0
01	1	1	0	1
11	1	1	0	1
10	0	0	1	0

$$B(t+1)_1 = B'CD + BC' + BD' \neq$$

C:

AB \ CD	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	0	1	0	1
10	0	1	0	1

$$C(t+1)_1 = C'D + CD'$$

D:

AB \ CD	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	0	0	1
10	1	0	0	1

$$D(t+1)_1 = D'$$

$X=1$

A:

AB \ CD	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	1	1	1

$$A(t+1)_2 = A'BC'D + AB + AD + AC$$

B:

AB \ CD	00	01	11	10
00	1	0	0	0
01	0	1	1	1
11	0	1	1	1
10	1	0	0	0

$$B(t+1)_2 = B'C'D' + BD + BC$$

C:

AB \ CD	00	01	11	10
00	1	0	1	0
01	1	0	1	0
11	1	0	1	0
10	1	0	1	0

$$C(t+1)_2 = CD + C'D'$$

D:

AB \ CD	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	0	0	1
10	1	0	0	1

$$D(t+1)_2 = D'$$

$$D_a = X'A(t+1)_1 + XA(t+1)_2$$

$$D_b = X'B(t+1)_1 + XB(t+1)_2$$

$$D_c = X'C(t+1)_1 + XC(t+1)_2$$

$$D_d = D'$$

ModelSim Simulation Results



Lab9-2 :

Main Code Screenshot

rgy_top.sv

```
V rgy_comb.sv V rgy_top.sv X V mcu.sv
de0_empty > design > V rgy_top.sv
1  module rgy_top(
2      input logic clk,
3      input logic reset,
4      output logic [1:0]r, g, y
5  );
6      logic [3:0]Q;
7
8      updown comb(
9          .clk(clk),
10         .reset(reset),
11         .x(1'b0),
12         .Q(Q)
13     );
14
15     rgy_comb rgy(
16         .q(Q),
17         .r(r),
18         .g(g),
19         .y(y)
20     );
21 endmodule
```

rgy_comb.sv

```
V rgy_comb.sv X V rgy_top.sv V mcu.sv
de0_empty > design > V rgy_comb.sv
1  module rgy_comb(
2      input logic [3:0]q,
3      output logic [1:0]r, g, y
4  );
5      assign r[0]=~q[3];
6      assign g[0] = q[3] & ~(q[2] & q[1]);
7      assign y[0]=q[1]&q[2]&q[3];
8      assign r[1]=q[3];
9      assign g[1] = ~q[3] & ~(q[2] & q[1]);
10     assign y[1]=q[1]&q[2]&~q[3];
11 endmodule
```

updown.sv same as the Lab1.

$X=0$

A:

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	1	1	0	1
10	1	1	1	1

$$A(t+1)_1 = A'BCD + AD' + AC' + AB'$$

B:

AB \ CD	00	01	11	10
00	0	0	1	0
01	1	1	0	1
11	1	1	0	1
10	0	0	1	0

$$B(t+1)_1 = B'CD + BC' + BD' \neq$$

C:

AB \ CD	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	0	1	0	1
10	0	1	0	1

$$C(t+1)_1 = C'D + CD'$$

D:

AB \ CD	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	0	0	1
10	1	0	0	1

$$D(t+1)_1 = D'$$

$X=1$

A:

AB \ CD	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	1	1	1

$$A(t+1)_2 = A'BC'D + AB + AD + AC$$

B:

AB \ CD	00	01	11	10
00	1	0	0	0
01	0	1	1	1
11	0	1	1	1
10	1	0	0	0

$$B(t+1)_2 = B'C'D' + BD + BC$$

C:

AB \ CD	00	01	11	10
00	1	0	1	0
01	1	0	1	0
11	1	0	1	0
10	1	0	1	0

$$C(t+1)_2 = CD + C'D'$$

D:

AB \ CD	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	0	0	1
10	1	0	0	1

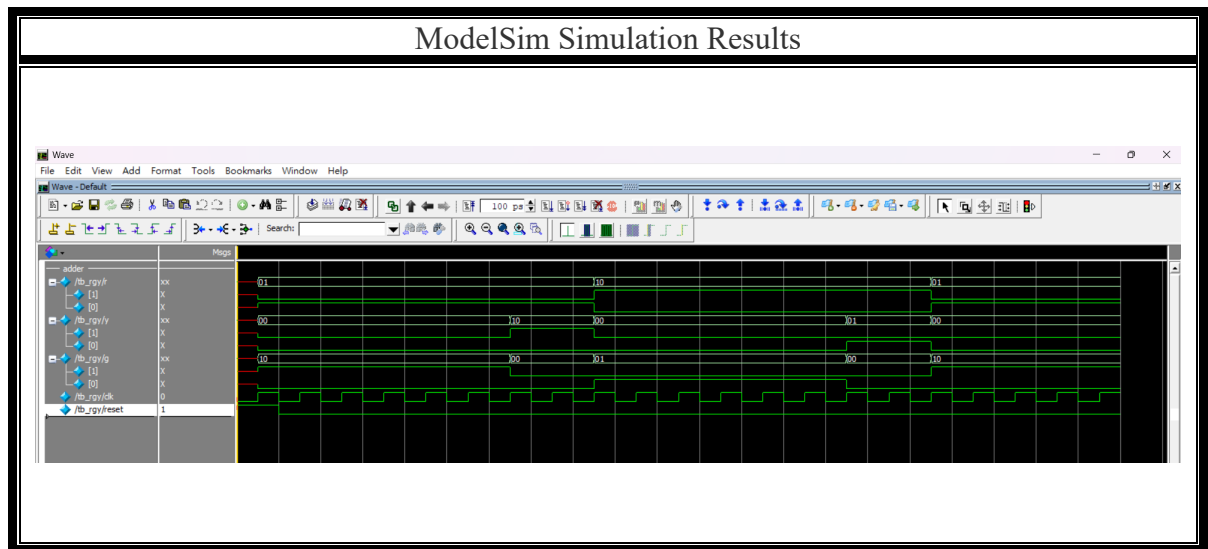
$$D(t+1)_2 = D'$$

$$D_a = X'A(t+1)_1 + XA(t+1)_2$$

$$D_b = X'B(t+1)_1 + XB(t+1)_2$$

$$D_c = X'C(t+1)_1 + XC(t+1)_2$$

$$D_d = D'$$



Lab Summary and Reflection :

Summary:

In this experiment, we leveraged Quartus and the Altera DE0 development board to realize two complementary sequential-logic systems in SystemVerilog: a 4-bit synchronous up/down counter and a dual traffic-light controller. For the counter, four D-flip-flops formed a register that, under the direction of an input signal, either incremented or decremented its 4-bit output. To make the count visually discernible on the seven-segment display, the on-chip 50 MHz clock was fed into a 32-bit register and its 23rd bit was tapped as a slow clock—effectively dividing the master clock by 2^{24} . Following Karnaugh-map minimization, the next-state logic was coded in the updown.sv, dfflipflop.sv, and freg_div_23.sv modules, and functional correctness was confirmed in ModelSim. On hardware, the seven-segment display reflected the count sequence in response to the switch-controlled up/down command and the reset button.

The traffic-light portion extended this framework by driving two separate red/green/yellow sets according to a 4-state machine. The rgy_top.sv and rgy_comb.sv modules decoded the counter's two least significant bits into the prescribed lighting pattern—traffic-light set #0 cycling through red→green→yellow→red and set #1 offset by one state, cycling green→yellow→red→green. Simulation waveforms verified that each light adhered to its timing, and pin assignments mapped those signals to the DE0 board's LEDs.

Reflection:

Through this lab, I deepened my grasp of synchronous sequential design, from state-diagram conception through hardware realization. Implementing the frequency divider underscored the importance of aligning simulation timescales with human-perceptible rates, teaching me how to choose appropriate tap bits for clock division. Deriving combinational logic via Karnaugh maps reinforced systematic minimization, yet translating those expressions in SystemVerilog reminded me to vigilantly confirm signal widths and case sensitivities to avoid subtle synthesis errors. Integrating multiple modules into a single Quartus project further illuminated the necessity of precise port declaration and pin mapping for successful FPGA deployment. Overall, this exercise not only solidified my HDL coding and verification workflow but also honed my ability to iterate between simulation and hardware testing to achieve reliable digital designs.